4.2.1. Phân vùng tương đương (Equivalence Partitioning - EP) 📊

Khái niệm cơ bản

Phân vùng tương đương (EP) chia dữ liệu thành các **phân vùng (partitions)** dựa trên giả định: *tất cả các phần tử trong cùng một phân vùng sẽ được xử lý giống nhau*.

Nguyên lý hoạt động 💡

Lý thuyết: Nếu một test case với giá trị từ phân vùng A phát hiện được lỗi → bất kỳ giá trị nào khác từ phân vùng A cũng sẽ phát hiện được lỗi đó.

Kết luận: **Một test case cho mỗi phân vùng là đủ** → giảm số lượng test cần thiết.

Ví dụ: Form nhập tuổi (18-65)

- Phân vùng hợp lệ: 18 ≤ tuổi ≤ 65
- Phân vùng không hợp lệ 1: tuổi < 18
- Phân vùng không hợp lệ 2: tuổi > 65

Bạn chỉ cần test: 30, 15, 70 → không cần test 18, 19, 20, 21...

Đặc điểm chính

1. Phạm vi áp dụng 🎯

Có thể xác định phân vùng cho bất kỳ phần tử dữ liệu nào:

Loại dữ liệu	Ví dụ
Đầu vào (Input)	Username, password, số điện thoại
Đầu ra (Output)	Mã trạng thái HTTP (200, 404, 500)
Mục cấu hình (Configuration)	Ngôn ngữ hiển thị (VI, EN, FR)
Giá trị nội bộ (Internal values)	Biến đếm, giá trị trong bộ nhớ
Giá trị thời gian (Time-related)	Thời gian timeout (5s, 30s, 60s)
Tham số giao diện (Interface parameters)	API parameters, query strings

2. Tính chất phân vùng

Các dạng phân vùng:

- **Liên tục** (continuous): 0.0 - 100.0 (số thực)
- **Rời rạc** (discrete): 1, 2, 3, 4, 5 (số nguyên)
- **Có thứ tự** (ordered): Thấp < Trung bình < Cao
- **Không thứ tự** (unordered): Đỏ, Xanh, Vàng
- **Hữu hạn** (finite): Tháng 1-12
- **Vô hạn** (infinite): Mọi số nguyên dương

Quy tắc quan trọng ⚠️ :

- Các phân vùng **KHÔNG được chồng lấn** (non-overlapping)
- Các phân vùng phải là **tập hợp không rỗng** (non-empty sets)

```
❌ SAI: Phân vùng 1: 1–50, Phân vùng 2: 50–100 (chồng lấn tại 50)
✅ ĐÚNG: Phân vùng 1: 1–49, Phân vùng 2: 50–100
```

Phân loại phân vùng

1. Phân vùng hợp lệ (Valid Partition) ✅

Chứa giá trị hợp lệ mà hệ thống nên xử lý bình thường.

Ví dụ: Số lượng sản phẩm (1-99)

- Test với: 1, 50, 99 → Cho phép đặt hàng

2. Phân vùng không hợp lệ (Invalid Partition) ❌

Chứa giá trị không hợp lệ mà hệ thống nên từ chối.

Ví dụ: Số lượng sản phẩm

- ≤ 0: Test với 0, -5 → Báo lỗi
- ≥ 100: Test với 150 → Báo lỗi
- Chữ cái: Test với "abc" → Báo lỗi
- Ký tự đặc biệt: Test với "@#" → Báo lỗi

> ⚠️ *Định nghĩa "hợp lệ/không hợp lệ" có thể khác nhau giữa các team.*

Đo lường độ bao phủ (Coverage)

Công thức tính 📐

```
Độ bao phủ (%) = (Số phân vùng đã test / Tổng số phân vùng) × 100%
```

Ví dụ:

- Tổng: 5 phân vùng (1 hợp lệ + 4 không hợp lệ)
- Đã test: 4 phân vùng
- Độ bao phủ: (4/5) × 100% = **80%**

Mục tiêu: Đạt 100% = test TẤT CẢ các phân vùng ít nhất 1 lần.

Độ bao phủ "Mỗi lựa chọn" (Each Choice Coverage)

Áp dụng cho: Hệ thống có **nhiều tham số đầu vào**.

Nguyên tắc: Mỗi phân vùng từ mỗi tham số được test **ít nhất 1 lần**, KHÔNG cần test mọi tổ hợp.

Ví dụ: Form đặt vé máy bay ✈️

Tham số 1 - Hạng vé:

- A1: Phổ thông
- A2: Thương gia
- A3: Hạng nhất

Tham số 2 - Độ tuổi:

- B1: Trẻ em (< 12)
- B2: Người lớn (12-64)
- B3: Người cao tuổi (≥ 65)

So sánh 2 cách tiếp cận

✅ **Each Choice Coverage (EP chuẩn)**

Số test cần thiết: 3 test cases

Test	Hạng vé	Độ tuổi	Cover
TC01	Phổ thông (A1)	Trẻ em (B1)	A1, B1 ✅
TC02	Thương gia (A2)	Người lớn (B2)	A2, B2 ✅
TC03	Hạng nhất (A3)	Cao tuổi (B3)	A3, B3 ✅

→ Tất cả 6 phân vùng đều được test với **chỉ 3 test cases**!

❌ **All Combinations (Không phải EP)**

Số test cần thiết: 3 × 3 = 9 test cases

Phải test: Phổ thông-Trẻ em, Phổ thông-Người lớn, Phổ thông-Cao tuổi, Thương gia-Trẻ em, Thương gia-Người lớn, Thương gia-Cao tuổi, Hạng nhất-Trẻ em, Hạng nhất-Người lớn, Hạng nhất-Cao tuổi.

→ Tốn thời gian và không cần thiết trong EP!

Tại sao Each Choice Coverage "đủ"? 💡

Giả định của EP: Các tham số xử lý **độc lập** với nhau.

Nếu hệ thống xử lý đúng:

- "Phổ thông" với "Trẻ em" ✅
- "Thương gia" với "Người lớn" ✅

→ Rất có khả năng sẽ xử lý đúng "Thương gia" với "Trẻ em"

Lợi ích: Với 5 tham số, mỗi tham số 3 phân vùng:

- Each Choice: ~5 test cases
- All Combinations: 3^5 = **243 test cases** 😱

Khi nào CẦN test tổ hợp? ⚠️

EP KHÔNG đủ khi các tham số có **tương tác phức tạp**.

Ví dụ: Khuyến mãi đặc biệt

- Trẻ em + Hạng nhất → Giảm 50%
- Cao tuổi + Phổ thông → Giảm 30%

- Các tổ hợp khác → Không giảm

→ Logic phụ thuộc vào **tổ hợp cụ thể** → phải test tổ hợp!

Kỹ thuật phù hợp: Pairwise Testing hoặc Decision Table Testing (không phải EP).

Tóm tắt

EP giúp:

- ✅ Giảm số lượng test cases hiệu quả
- ✅ Dễ đo lường độ bao phủ
- ✅ Phù hợp khi tham số xử lý độc lập

Lưu ý:

- ⚠️ Cần hiểu rõ logic xử lý của hệ thống
- ⚠️ Each Choice Coverage = EP chuẩn (không test mọi tổ hợp)
- ⚠️ Nếu tham số có tương tác phức tạp → dùng kỹ thuật khác

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.